

AGUAFRESS

Stack Tecnológico — Versionado y Justificación

Arquitectura de Microservicios v1

Documento técnico — Proyecto AguaFress AG-82

1. Introduccion

El presente documento define el stack tecnologico del sistema AguaFress, una plataforma de distribucion de agua con arquitectura de microservicios. Cada tecnologia fue seleccionada en funcion de los requerimientos especificos del sistema: soporte para usuarios web y mobile, procesamiento distribuido de pedidos, sincronizacion de eventos entre servicios y escalabilidad independiente por modulo.

Este documento responde a la tarea AG-82 (Confirmar stack y justificar cada tecnologia) y complementa el diagrama de arquitectura AG-81.

2. Resumen del Stack

La siguiente tabla consolida todas las tecnologias con su version confirmada y la justificacion orientada al proyecto:

Categoria	Tecnologia	Justificacion orientada al proyecto
Backend	Node.js 20 LTS	Estabilidad LTS para sistema de microservicios con alta concurrencia
Backend	NestJS 10.x	Arquitectura de microservicios, API Gateway, comunicacion TCP y Redis
Backend	TypeScript 5.x	Consistencia entre servicios y seguridad de tipos en DTOs distribuidos
Backend	Prisma ORM 5.x	Interaccion rapida y segura con bases de datos relacionales (users, orders)
Frontend Web	React 18.x + TS 5.x	Interfaz dinamica: catalogo, carrito y seguimiento de pedidos en tiempo real
Frontend Mobile	React Native 0.74+	Pedidos desde dispositivos moviles, reutilizando logica del frontend web
Base de Datos	PostgreSQL 15.x	Transacciones criticas: usuarios, pedidos y productos con integridad fuerte

Categoría	Tecnología	Justificación orientada al proyecto
Base de Datos	MySQL 8.0.x	Reportes y metricas sin impactar la base transaccional principal
Base de Datos	MongoDB 6.x	Auditoria y logs con estructura flexible sin afectar otros servicios
Comunicación	Redis 7.x Pub/Sub	Eventos asincronicos: pedidos, notificaciones, sincronización entre MS
Comunicación	TCP NestJS	Operaciones sincronicas que requieren respuesta inmediata (validaciones)
Infraestructura	Docker 24.x	Contenerización de cada microservicio para consistencia entre entornos
Infraestructura	Docker Compose v2.x	Orquestación simple de todos los servicios en desarrollo
Infraestructura	Nginx 1.24.x	Reverse proxy, centralización de acceso, HTTPS y optimización de conexiones
Seguridad	JWT jsonwebtoken 9.x	Autenticación stateless para web y mobile con acceso seguro al sistema
Seguridad	bcrypt 5.x	Hashing seguro de credenciales de usuarios
Seguridad	HTTPS TLS 1.2/1.3	Protección del tráfico entre clientes y backend gestionada por Nginx

3. Backend

3.1 Node.js 20 LTS

Version LTS seleccionada para garantizar estabilidad y continuidad durante todo el ciclo de desarrollo del proyecto. Su modelo de E/S no bloqueante resulta esencial para un sistema que maneja múltiples pedidos y eventos de forma concurrente sin degradación de rendimiento.

3.2 NestJS 10.x

Framework principal del backend. Se elige porque el sistema requiere:

- Un API Gateway centralizado que enruta y autentica todas las peticiones
- Comunicación sincronica via TCP entre el gateway y los microservicios
- Comunicación asincronica via Redis Pub/Sub para propagación de eventos
- Estructura modular compatible con el patron de 7 microservicios independientes

NestJS 10.x integra soporte nativo para ambas estrategias de comunicación sin librerías adicionales, lo que reduce la complejidad del proyecto.

3.3 TypeScript 5.x

Utilizado en todos los servicios del backend. En una arquitectura distribuida, la consistencia de los contratos de datos entre servicios (DTOs) es crítica para evitar errores en tiempo de ejecución. TypeScript strict mode garantiza que los tipos sean validados en tiempo de compilación antes de que lleguen a producción.

3.4 Prisma ORM 5.x

Elegido para los servicios que interactúan con bases de datos relacionales (Usuarios, Productos, Carrito, Pedidos, Entregas). Prisma genera clientes tipados a partir del schema, lo que elimina errores de consulta y acelera el desarrollo. Su integración nativa con TypeScript lo hace especialmente adecuado para este proyecto.

4. Frontend — Web y Mobile

4.1 React 18.x + TypeScript 5.x (Web)

Se utiliza para construir la interfaz de usuario web que permite:

- Navegar el catálogo de productos por vendedor
- Gestionar el carrito de compras
- Realizar y monitorear el estado de pedidos en tiempo real

React 18 incorpora mejoras de renderizado concurrente que optimizan la experiencia en listas de productos y actualizaciones de estado dinámicas.

4.2 React Native 0.74+ + TypeScript 5.x (Mobile)

El sistema fue concebido para ser utilizado principalmente desde dispositivos móviles, donde los clientes realizan pedidos desde cualquier ubicación. React Native permite:

- Reutilizar la lógica de negocio y hooks del frontend web
- Mantener coherencia funcional entre plataformas
- Reducir el tiempo de desarrollo al compartir código entre web y mobile

Ambos clientes consumen la misma API bajo un enfoque API-first, garantizando consistencia total entre plataformas. Los tokens JWT se envían en los headers de Authorization tanto en Axios (web) como en Fetch (mobile).

5. Persistencia Poliglota

El sistema implementa persistencia poliglota: cada servicio utiliza el motor de base de datos más adecuado para su tipo de dato y patrón de acceso.

Motor	Version	Servicio que lo usa	Razon de eleccion
PostgreSQL	15.x	Usuarios, Productos, Carrito, Pedidos, Entregas	Consistencia fuerte, integridad referencial, transacciones ACID

Motor	Version	Servicio que lo usa	Razon de eleccion
MySQL	8.0.x	Metricas Service	Lectura optimizada para reportes sin impactar servicios transaccionales
MongoDB	6.x	Auditoria Service	Esquema flexible para logs y eventos del sistema con alta variabilidad

5.1 PostgreSQL 15.x — Base transaccional principal

Utilizado por los servicios criticos del sistema (Usuarios, Productos, Carrito, Pedidos, Entregas). Se requiere:

- Consistencia fuerte en operaciones de creacion de pedidos
- Integridad referencial entre usuarios, productos y pedidos
- Transacciones ACID para operaciones de pago y confirmacion de entregas

La version 15 incorpora mejoras de rendimiento y particionamiento que mejoran el procesamiento de consultas concurrentes, algo critico durante picos de pedidos.

5.2 MySQL 8.0.x — Metricas y reportes

Empleado exclusivamente en el Metricas Service para generar reportes de ventas por vendedor y productos mas vendidos. Separar este motor evita que las consultas de lectura intensiva de reportes impacten el rendimiento de la base transaccional PostgreSQL principal.

5.3 MongoDB 6.x — Auditoria flexible

Utilizado por el Auditoria Service para registrar eventos del sistema (acciones de usuarios, cambios de estado, logs). Su esquema flexible permite almacenar eventos con estructuras variables sin requerir migraciones. La coleccion audit_logs crece de forma independiente sin afectar a los demas servicios.

6. Comunicacion entre Servicios

6.1 Redis 7.x — Pub/Sub asincronico

Se utiliza como bus de eventos asincronicos entre microservicios. Los principales eventos del sistema propagados via Redis son:

- UserCreated / UserUpdated → propagados al resto del sistema
- ProductUpdated / ProductDeleted → actualizan product_cache en Pedidos
- OrderCreated / OrderDelivered → recibidos por Auditoria y Metricas

Redis garantiza que si un microservicio esta caido, los mensajes no se pierden (Persistencia AOF). Ademas se usa como cache de sesiones y cache de productos, reduciendo la carga sobre las bases de datos.

6.2 TCP NestJS ClientProxy — Sincrono

Se implementa en operaciones donde el API Gateway requiere una respuesta inmediata del microservicio correspondiente. El patron de comunicacion es request / response directo entre el Gateway y el servicio destino, sin intermediarios. Se usa para validaciones, consultas de estado y operaciones que no pueden resolverse de forma asincronica.

7. Infraestructura

7.1 Docker 24.x

Cada uno de los 7 microservicios se conteneriza de forma independiente. Esto garantiza:

- Consistencia entre entorno de desarrollo, testing y produccion
- Escalabilidad independiente por servicio segun carga
- Aislamiento de dependencias entre servicios

7.2 Docker Compose v2.x

Permite levantar y orquestar todos los servicios del sistema con un unico comando en entornos de desarrollo. Facilita la colaboracion del equipo al eliminar configuraciones manuales por maquina.

7.3 Nginx 1.24.x — Reverse Proxy

Actua como capa de entrada al sistema, previa al API Gateway. Sus responsabilidades son:

- Centralizar el acceso de clientes web y mobile en un unico punto de entrada
- Gestionar HTTPS con terminacion TLS 1.2/1.3 (certificados SSL)
- Mejorar el rendimiento mediante manejo eficiente de conexiones HTTP

En un sistema con multiples clientes y microservicios, Nginx es la capa previa indispensable antes de que las peticiones lleguen al API Gateway de NestJS.

8. Seguridad

8.1 JWT — jsonwebtoken 9.x

Autenticacion stateless implementada en el API Gateway. Los tokens JWT son generados al iniciar sesion y validados en cada peticion sin necesidad de consultar la base de datos, lo que reduce la latencia. Soporta los tres roles del sistema: ADMIN, VENDEDOR y CLIENTE, con aislamiento por vendedor_id.

8.2 bcrypt 5.x

Utilizado en el Usuarios Service para hashear las contraseñas antes de almacenarlas. Garantiza que incluso si la base de datos fuese comprometida, las credenciales originales no puedan ser recuperadas.

8.3 HTTPS — TLS 1.2 / 1.3

Toda la comunicacion entre los clientes (web y mobile) y el backend viaja cifrada. La terminacion TLS es gestionada por Nginx, que actua como proxy SSL antes de redirigir las peticiones internamente al API Gateway en HTTP.

9. Conclusion

El stack tecnologico de AguaFress fue definido en funcion de los requerimientos especificos del sistema:

- Web y mobile → React 18 + React Native bajo enfoque API-first
- Microservicios modulares → NestJS 10 con comunicacion TCP y Redis
- Persistencia diferenciada → PostgreSQL (transaccional), MySQL (reportes), MongoDB (auditoria)
- Eventos asincronicos → Redis 7 Pub/Sub con persistencia AOF
- Acceso centralizado y seguro → Nginx + JWT + HTTPS
- Escalabilidad e independencia → Docker + Docker Compose

Documento generado para el proyecto AguaFress — 2025